

Assignment 02

Submission deadline: May 18, 2026.

This assignment has two parts:

- Part A: an essay on Linux systems administration ()
- Part B: a codebase review pipeline using GNU Make and `ask` script

Part A - Administrative Roles and Responsibilities in Linux Systems (60 points)

Prepare an in-depth report that explores the administrative roles and responsibilities in Linux systems. Your report should cover the following key areas:

1. Key Administrative Tasks:

- Describe the main duties and responsibilities of a Linux system administrator.
- Identify and explain common tasks performed by Linux system administrators, such as user management, process control, file system management, and system monitoring.

2. Security and Compliance Responsibilities:

- Describe the security-related responsibilities of a Linux system administrator, including user authentication, authorization, and system hardening.

3. System Maintenance and Automation:

- Discuss the various maintenance tasks performed by Linux system administrators, including patch management, software updates, backup operations, and automation tools (such as cron jobs, shell scripts) to streamline administrative tasks.
- Explain the importance of log files.

Additional Guidelines:

- Include references to support your points.
- Use clear language and organize your essay with headings and subheadings.
- Ensure your essay is well-structured and logically organized.

Submission Instructions:

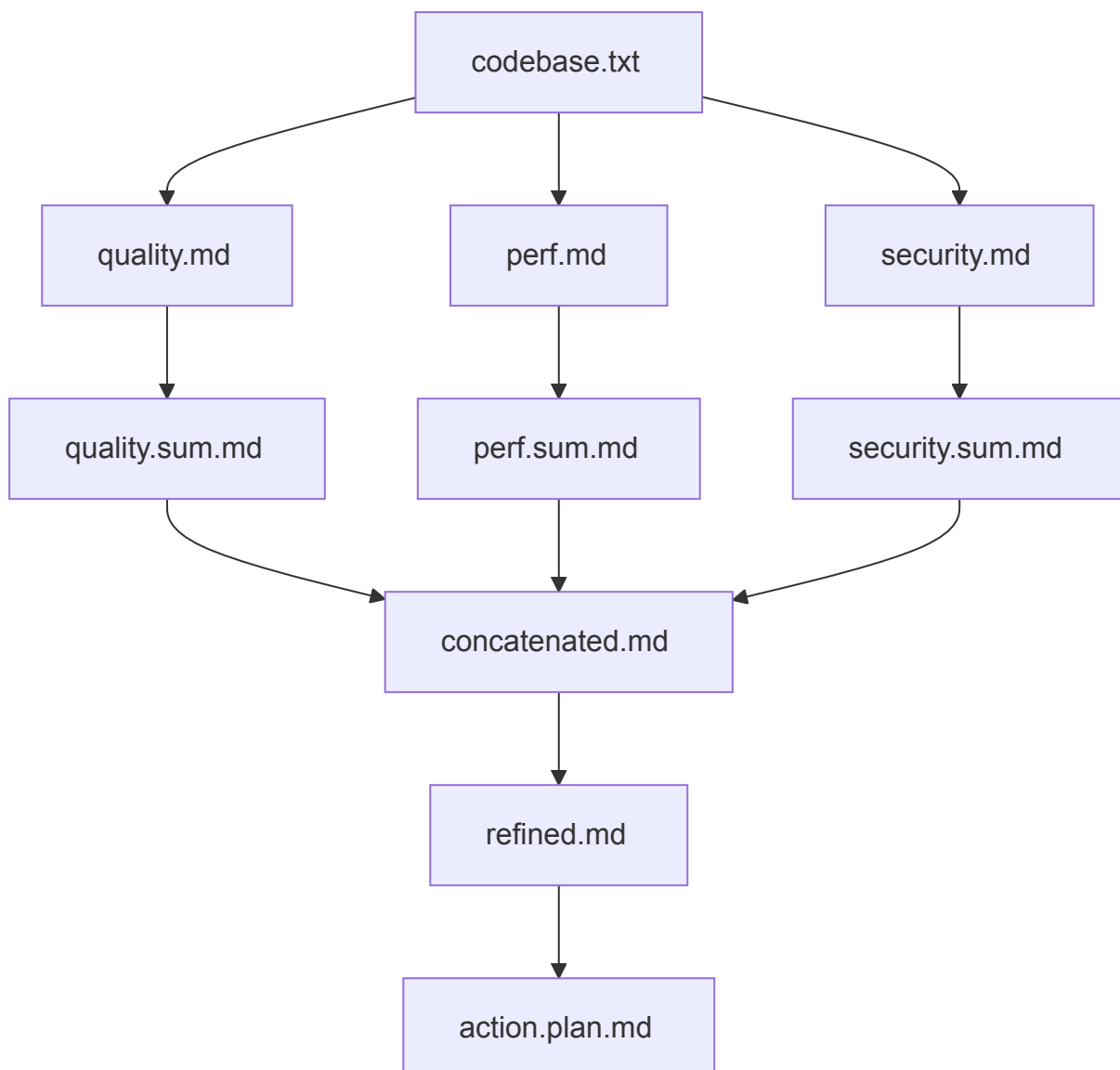
- Submit your essay as a PDF file through Blackboard platform.
 - Include your name, student ID, and course information on the first page.
 - The submission deadline is May 10, 2024, at 23:59.
-

Part B - Codebase Review and Engineering Action Plan (Make + ask Pipeline) (40 points)

You are given a single file `codebase.txt` containing a codebase or module. Create a parallel LLM pipeline with `GNU Make` and the `ask` script from the previous homework to generate a final engineering action plan which is in markdown file format.

The pipeline takes one code file and turns it into a clear action plan by splitting the work into parallel steps. It runs three analyses at the same time (quality, performance, security), then summarizes each result to keep things concise. These summaries are combined into a simple report, which is then merged and refined into a final prioritized action plan. Using `GNU Make` ensures everything runs in the right order and in parallel, making the process faster and more structured.

Execution flow for engineering action plan:



Update your first Git repository where the `ask` script is included. If you haven't submitted your first assignment, check the `ask` script attached to the assessment. You may also find a sample `codebase.txt` as an attachment.

You must use:

`ask` (stdin → LLM → stdout)

`make` (DAG orchestration)

`make -j` for parallel execution

When you run the `make -j` command, all the files in the diagram above should be created in that specific order.

Follow the instructions below when writing prompts and commands in your `Makefile`.

Phase 1 — FAN-OUT (parallel)

Create **3 parallel targets**:

1. Code Quality

- readability, structure, duplication
- output: 5–7 bullets (problem → fix)

2. Performance

- bottlenecks, inefficiencies
- output: 5–7 bullets (issue → optimization)

3. Security

- vulnerabilities, unsafe patterns
- output: 5–7 bullets (risk → mitigation)

Phase 2 — LOCAL SUMMARIZATION

For each branch:

- compress to **exactly 5 bullets**
- keep only actionable items

Phase 3 — CONCAT REPORT (no LLM)

Concatenate all summaries into a single file **with headings**:

Report structure:

```
## Code Quality
```

```
<quality.sum>
```

```
## Performance
```

```
<perf.sum>
```

```
## Security
<security.sum>
```

- This step must be done using shell tools (e.g., `cat` , `echo`) in Make
- Do NOT use `ask` here

Phase 4 — FAN-IN #1 (LLM Refine)

Input: Concat file

Sections:

- Code Quality
- Performance
- Security

Rules:

- remove duplicates
- keep high-signal issues only

Phase 5 — FAN-IN #2

Input: Concat file

Generate final output:

“Engineering Action Plan”

Must include:

- prioritized actions (High / Medium / Low)
- effort estimate (Small / Medium / Large)
- execution order

Submission

Submission deadline: May 18, 2026.

Submit one PDF file to Blackboard.

The first page must include:

1. Your name and student ID
2. A `git clone` command that downloads your project into a folder named in this format:
`<student_id>_<full_name>`

Example:

```
git clone "https://github.com/johndoe/makefile.git"  
"202306001234_John_Doe"
```

The rest of the PDF must include:

- Part A: Essay
- Part B:
 - Explain how the pipeline works for the three cases below:
 - when `codebase.txt` is updated
 - when `security.sum.md` is updated
 - when `refined.md` is updated
 - Include your `Makefile` file content inside the PDF document, not as a separate attachment